

**National Technical University of Ukraine «Igor Sikorsky Kyiv
Polytechnic Institute»**

**Educational and Scientific Institute of Atomic and Thermal Energy
Department of Digital Technologies in Energy**

Calculation and graphics work

in the discipline «Visualization of graphical and geometric information»

Topic: «Operations on texture coordinates»

Variant 3 (scaling)

Performed by
student of the TR-51mp group
Barbar Vladyslav Volodymyrovych

Checked by
Associate Professor, PhD
Demchyshyn Anatoliy Anatoliyovych

Kyiv 2025

I. TASK DESCRIPTION

The primary objective of this laboratory work is to deepen the understanding of texture mapping techniques in 3D computer graphics using the WebGL API. The work focuses on the manipulation of texture coordinates to achieve dynamic visual effects on parametric surfaces. Specifically, the task requires the development of a software application that renders a textured 3D model of a «Pear Surface,» which was implemented in previous assignments. The application must support advanced texturing features, including diffuse, specular, and normal mapping, to ensure a high level of visual realism.

For Variant 3, the specific individual task involves the implementation of texture scaling functionality. Unlike simple scaling, which typically occurs relative to the origin of the texture coordinate system (0, 0), this assignment requires the implementation of scaling relative to an arbitrary, user-defined point on the surface (the center of scaling). This feature allows users to «zoom in» or «zoom out» on a specific detail of the texture rather than simply expanding the texture from the corner.

Furthermore, the application must provide an interactive mechanism for moving this center of scaling across the surface of the object. The user must be able to control the position of the center point using the keyboard keys (W, A, S, D). To improve usability and visual feedback, the application is required to visualize this center point directly on the 3D surface. A visual marker, such as a small sphere, must track the current position of the texture center, providing the user with a clear indication of the point around which the scaling transformation is being applied. Finally, the implementation must handle texture wrapping correctly, ensuring that the texture repeats seamlessly when the scaling factor leads to coordinate values outside the standard range.

II. THEORETICAL BACKGROUND

2.1. Texture Mapping Fundamentals

Texture mapping is a fundamental concept in computer graphics that involves mapping a two-dimensional image, known as a texture, onto a three-dimensional surface. This process is governed by texture coordinates, commonly denoted as (u, v). These coordinates act as an anchor, relating a specific point on the 3D mesh to a corresponding

pixel (texel) on the 2D texture image. The coordinate system for textures typically ranges from 0.0 to 1.0 on both axes, where (0, 0) represents the bottom-left corner and (1, 1) represents the top-right corner of the image. For parametric surfaces, such as the Pear surface used in this work, the (u, v) coordinates are often derived directly from the parametric variables used to generate the geometry. For instance, the parameter corresponding to the angle of rotation around the axis is mapped to the u coordinate, while the parameter corresponding to the height or profile of the object is mapped to the v coordinate. This direct mapping ensures that the texture wraps naturally around the curvature of the object without significant distortion.

2.2. Mathematics of Texture Scaling

Scaling texture coordinates is a transformation that changes the density of the texture on the surface. Mathematically, scaling is achieved by multiplying the texture coordinates by a scaling factor. However, a straightforward multiplication scales the coordinates relative to the origin (0, 0). This means that as the scale increases, the texture appears to shrink towards the bottom-left corner. To implement scaling relative to an arbitrary center point $C(u_c, v_c)$, a sequence of linear transformations is required. First, the coordinate system must be translated such that the center point C becomes the new origin. This is done by subtracting the center coordinates from the current texture coordinates. Second, the scaling factor is applied to these translated coordinates. Finally, the system is translated back to its original position by adding the center coordinates. This sequence ensures that the texture expands or contracts specifically around the chosen point C , keeping that point visually stationary while the surrounding texture moves.

2.3. Texture Wrapping Modes

When texture coordinates are scaled or transformed, their values often exceed the standard [0, 1] range. The behavior of the graphics pipeline in these scenarios is determined by the texture wrapping mode. One common mode is `GL_CLAMP_TO_EDGE`, which constraints the coordinates to the [0, 1] range. If coordinates exceed this range, the color of the edge texel is repeated, resulting in a «streaked» appearance. This mode is generally unsuitable for scaling tasks where the texture is expected to tile. The alternative mode, `GL_REPEAT`, treats the integer part of

the coordinate as the number of repetitions and uses the fractional part to sample the texture. This creates a seamless tiling effect, which is essential for this laboratory work to allow for arbitrary scaling factors greater than 1.0 without visual artifacts.

2.4. Inverse Mapping for Visualization

To visualize the center of scaling on the 3D surface, an inverse mapping process is required. The scaling logic operates in the 2D texture space (u, v), but the visual marker must be placed in the 3D world space (x, y, z). Since the surface is defined parametrically, it is possible to calculate the 3D position corresponding to any given (u, v) pair. The v coordinate is linearly mapped to the range of the z parameter (height) of the pear, while the u coordinate is mapped to the rotation angle θ . Using the mathematical definition of the Pear surface, the radius r at a given height z is calculated first. Then, using polar coordinates, the x and y positions are derived as $x = r \cdot \cos(\theta)$ and $y = r \cdot \sin(\theta)$. This calculation allows the application to dynamically update the transformation matrix of the marker model, ensuring it always stays attached to the surface at the correct location.

III. IMPLEMENTATION DETAILS

3.1. Shader Development

The core logic for texture coordinate transformation is implemented within the Vertex Shader to ensure high performance. Standard vertex shaders typically pass texture coordinates directly to the fragment shader. In this implementation, however, the vertex shader was modified to accept two additional uniform variables: a floating-point value `u_TexScale` representing the scaling factor, and a 2D vector `u_TexCenter` representing the center of scaling. Before passing the coordinates to the fragment shader, the vertex shader applies the translation-scaling-translation formula discussed in the theoretical section. This ensures that the interpolation of texture coordinates across the triangle primitives reflects the scaling transformation correctly. The fragment shader remains largely responsible for the Phong lighting model, using the transformed coordinates to sample from the diffuse, specular, and normal maps.

3.2. JavaScript Application

The main application logic is contained within the `main.js` file, which manages the WebGL context, state variables, and event listeners. The state of the application includes variables for the current scaling factor and the (u, v) coordinates of the texture center. Event listeners are attached to the HTML document to capture user input. Specifically, a `keydown` event listener monitors the W, A, S, and D keys. When a key is pressed, the corresponding component of the texture center variable is incremented or decremented by a small step value. Bounds checking is implemented to ensure the coordinates remain within the $[0, 1]$ range. Simultaneously, an input event listener on the UI slider updates the scaling factor. Upon any change in state, the `draw` function is called to re-render the scene with the updated parameters.

3.3. Marker Visualization

To assist the user in navigating the surface, a visual marker was implemented as a separate 3D object. A helper function `CreateSphere` was added to the `Model.js` module, which generates the vertex data for a small sphere. During the rendering loop, the application calculates the 3D position of the texture center using the parametric equations of the Pear surface. A translation matrix is constructed based on these calculated coordinates and is multiplied by the view matrix of the scene. This combined matrix is then passed to the shader when rendering the marker. To make the marker distinct from the rest of the scene, its material properties are set to have a high ambient red component and zero diffuse or specular reflection, making it appear as a solid, glowing red dot.

3.4. Texture Configuration

A critical fix was applied to the `TextureHandler.js` file to support correct tiling. The initial implementation utilized the `CLAMP_TO_EDGE` wrapping parameter, which caused visual artifacts when the scaling factor exceeded 1.0. This was corrected by changing the texture parameters `TEXTURE_WRAP_S` and `TEXTURE_WRAP_T` to `REPEAT`. This change instructs the WebGL implementation to tile the texture image infinitely in all directions, allowing for seamless scaling and movement of the texture coordinates. Additionally, mipmapping was enabled with trilinear filtering

(`LINEAR_MIPMAP_LINEAR`) to prevent aliasing artifacts and texture shimmering when the texture is scaled down or viewed at oblique angles.

IV. USER INSTRUCTION

4.1. General Interface Overview

Upon launching the application in a web browser, the user is presented with a 3D view of the textured Pear surface against a dark background. On the right side of the screen, a control panel allows for the adjustment of various parameters. The main viewport supports interactive rotation; the user can click and drag the mouse cursor (or use a finger on touch devices) to rotate the object and inspect it from different angles. This orbital camera control is essential for viewing the effects of normal mapping and specular highlights as the lighting angle changes relative to the surface (figure 1).

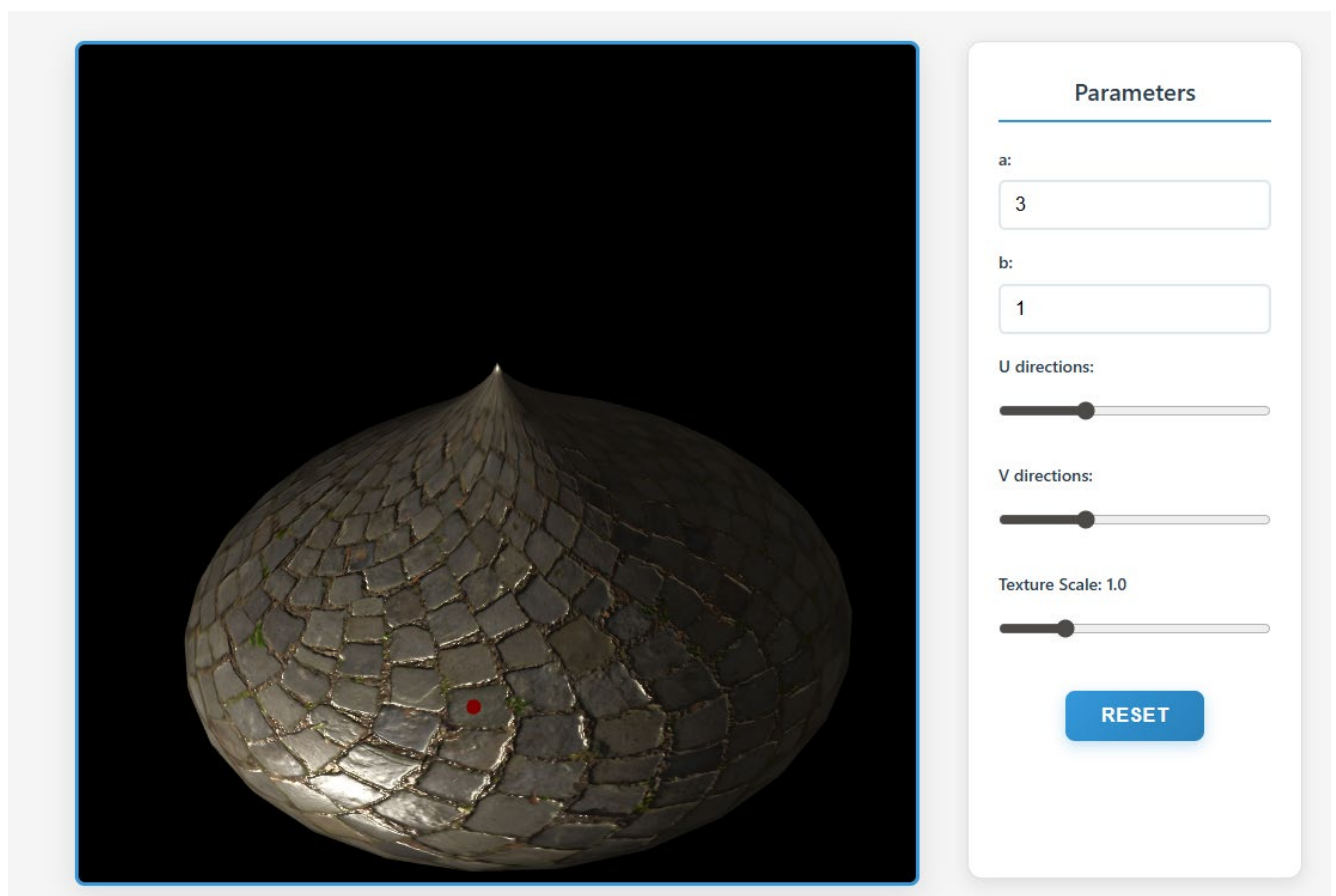


Figure 1 – General view of the application with the default scale

4.2. Controlling Texture Scaling

The scaling functionality is controlled via a slider labeled «Texture Scale» located in the control panel. By dragging the slider handle to the right, the user increases the scaling factor, causing the texture pattern (the cobblestones) to appear smaller and more repetitive. Dragging the slider to the left decreases the scaling factor, making the texture pattern larger. A numerical indicator next to the label displays the precise value of the current scaling factor (figure 2).

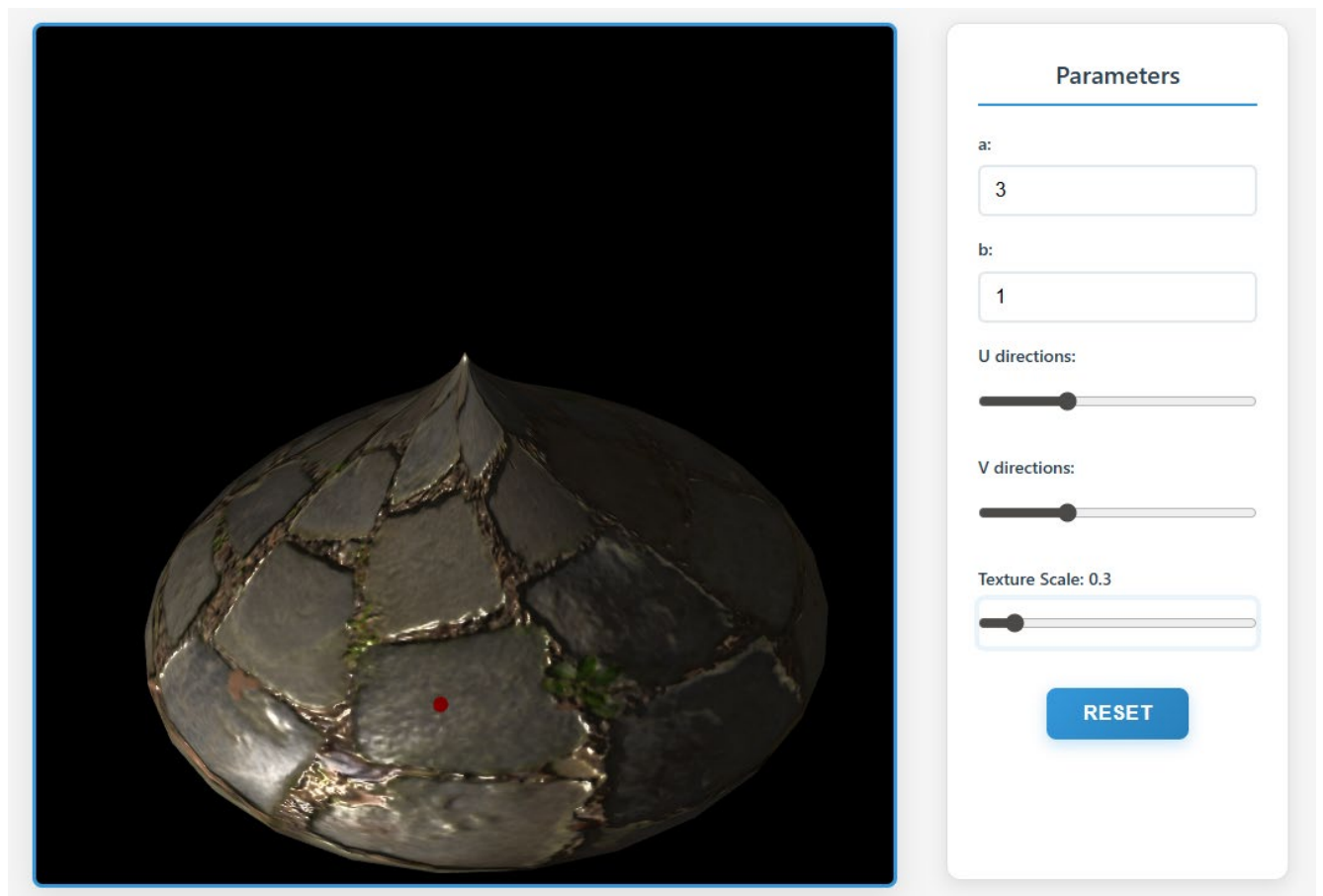


Figure 2 – The result of increasing the texture scale to 0.3

4.3 Moving the Center of Scaling

The unique feature of this application is the ability to move the focal point of the scaling operation. This is controlled via the keyboard using the standard WASD layout. Pressing the «W» key moves the center point upwards along the surface, while the «S» key moves it downwards. The «A» and «D» keys move the center point to the left and right around the object's circumference, respectively. As the user presses these keys, a

small red spherical marker moves across the surface of the pear. This marker indicates the exact geometric point that corresponds to the center of the texture scaling transformation. When the texture scale is adjusted, the texture will appear to expand or contract towards or away from this red marker (figure 3).

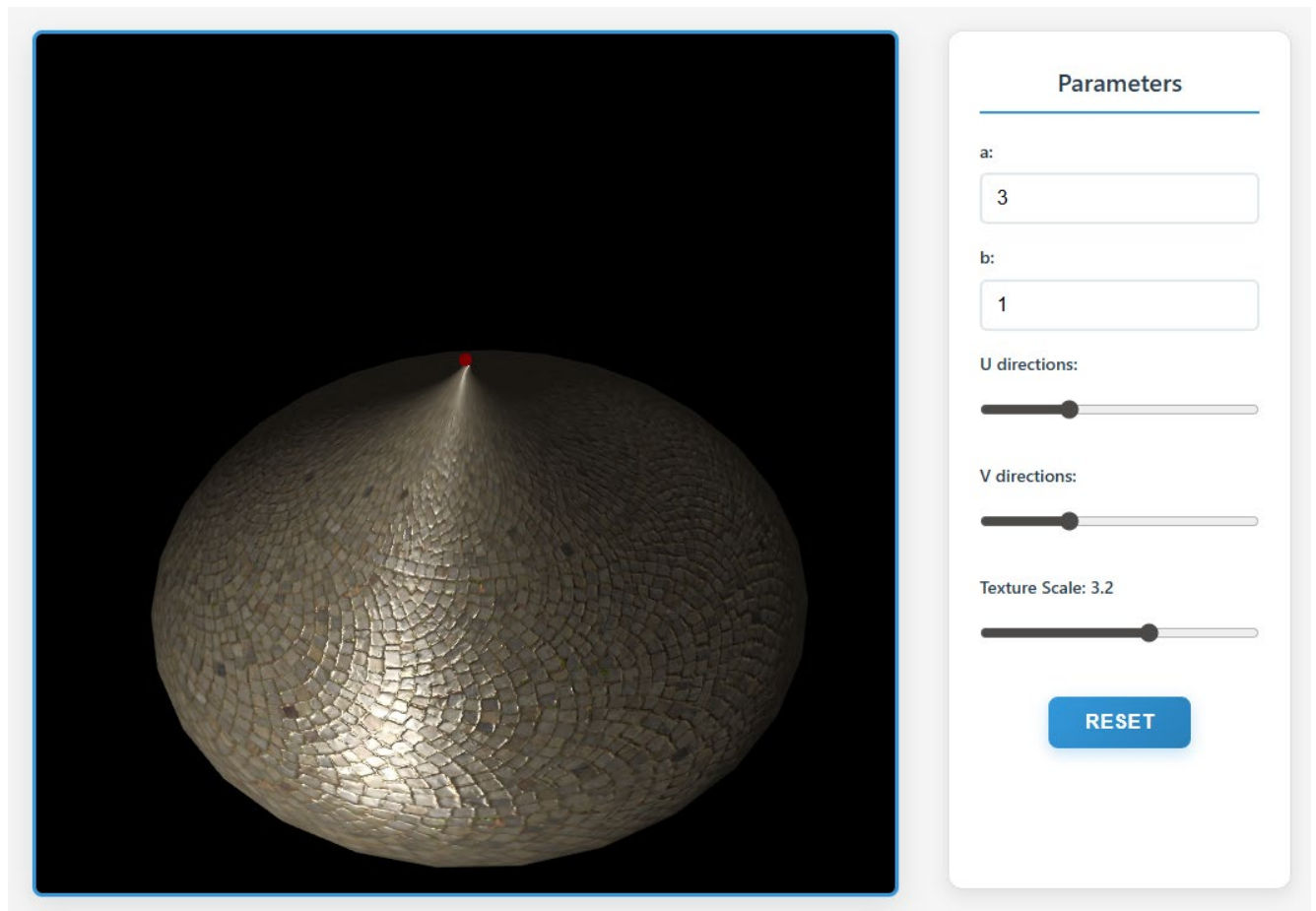


Figure 3 –The texture pattern converging towards the marker (top of the object)

V. SOURCE CODE SAMPLES

Listing 1. Vertex Shader (shader.gpu)

This listing demonstrates the GLSL code responsible for the transformation of texture coordinates. It shows the declaration of the new uniform variables and the mathematical operations applied to the vertex attributes

```
// Vertex shader
const vertexShaderSource = `
    attribute vec3 a_VertexPosition;
    attribute vec3 a_VertexNormal;
```



```

attribute vec3 a_VertexTangent;
attribute vec2 a_TexCoord;

uniform mat4 u_ProjectionMatrix;
uniform mat4 u_ModelViewMatrix;
uniform mat4 u_NormalMatrix;

// New uniforms for interactive scaling
uniform float u_TexScale; // Scaling factor
uniform vec2 u_TexCenter; // Center of scaling (u, v)

varying vec2 v_TexCoord;
varying vec3 v_LightPosTS;
varying vec3 v_ViewPosTS;
varying vec3 v_FragPosTS;

uniform vec3 u_LightPosition;

void main() {
    gl_Position = u_ProjectionMatrix * u_ModelViewMatrix * vec4(a_VertexPosition, 1.0);

    // Texture coordinate transformation logic
    // 1. Shift origin to the center of scaling
    vec2 centered = a_TexCoord - u_TexCenter;
    // 2. Apply scaling factor
    vec2 scaled = centered * u_TexScale;
    // 3. Shift origin back
    v_TexCoord = scaled + u_TexCenter;

    // ... (Tangent space calculation code omitted for brevity) ...
}
';

```

Listing 2. Keyboard Event Handling (main.js)

The following JavaScript snippet illustrates how keyboard events are captured and used to update the state of the texture center coordinates. It also shows the logic for clamping values and triggering a scene redraw.

```

// Keyboard event listener for WASD controls
window.addEventListener('keydown', (e) => {
  const step = 0.02; // Movement step size
  switch (e.code) {
    case 'KeyA':
      textureCenter[0] += step; // Move Left
      break;
    case 'KeyD':
      textureCenter[0] -= step; // Move Right
      break;
    case 'KeyW':
      textureCenter[1] -= step; // Move Up (inverted for intuitive control)
      break;
    case 'KeyS':
      textureCenter[1] += step; // Move Down
      break;
    default:
      return; // Ignore other keys
  }
  // Clamp coordinates to the [0, 1] range
  textureCenter[0] = Math.max(0.0, Math.min(1.0, textureCenter[0]));
  textureCenter[1] = Math.max(0.0, Math.min(1.0, textureCenter[1]));

  draw(); // Re-render the scene
});

```

Listing 3. Visual Marker Positioning (main.js)

This section of the code demonstrates the inverse mapping algorithm used to calculate the 3D position of the red marker based on the current texture coordinates.

```

// Calculate 3D position for the visual marker
let a = parseFloat(a_input.value);
let b = parseFloat(b_input.value);

// Retrieve current center coordinates
let u_tex = textureCenter[0];
let v_tex = textureCenter[1];

```

```

// Map v-coordinate to Z-height
let z_point = v_tex * a;

// Calculate Radius at this Z using the surface equation
const RZ = (z, a, b) => (z * Math.sqrt(z * (a - z))) / b;
let r_point = (z_point >= a) ? 0 : RZ(z_point, a, b);

// Map u-coordinate to rotation angle
let angle_point = (1.0 - u_tex) * 2 * Math.PI;

// Convert polar to Cartesian coordinates
let x_point = r_point * Math.sin(angle_point);
let y_point = r_point * Math.cos(angle_point);

// Apply translation to the marker model matrix
let translateMarker = m4.translation(x_point, y_point, z_point);
let markerMatrix = m4.multiply(matAccum1, translateMarker);

gl.uniformMatrix4fv(shProgram.iModelViewMatrix, false, markerMatrix);

```